ORBITAL FUTURE DECISION ENGINE

Nakshatra Final Hackathon — Round 2 Preparation Playbook

Team Fantastic 4

Anuj Singh · Sarthak Gavnekar · Nitin Chherke · Chirag Lalwani

Event details shown in the shared screenshot: 24 October 2026 · 12-hour offline final · MITS Campus, Gwalior

Purpose: explain the problem, show concrete examples, describe a defensible solution, compare it with existing approaches, plan the software build, and provide a master prompt for Claude.

Important scope note: The exact final-round rubric, pitch duration, submission format, allowed tools and detailed organizer instructions were not shown. Confirm them officially. This guide does not assume every proposed feature is already implemented in the current prototype.

1. Executive summary

Orbital Future Decision Engine (OFDE) is a research decision-support concept for orbital conjunction and space-debris scenarios. Its central idea is to help a user compare more than one plausible future, inspect uncertainty, and understand how changed assumptions affect an outcome. The goal is an explainable workflow—not replacing operational collision-avoidance systems.

Recommended MVP: select a clearly labelled public or synthetic sample; validate the inputs; run only verified calculations; compare a baseline with a controlled alternative; explain the difference and assumptions; display uncertainty only when the implemented method supports it; provide a reproducible summary.

The current website is a static research/concept experience with interactive illustrations and controls. Before changing code, audit which features are real calculations, which are visual demonstrations, and which are not implemented.

Do not claim live orbital monitoring, validated collision probabilities, executed Monte Carlo experiments, autonomous maneuver recommendations, or scientific accuracy unless the implementation, data and tests genuinely support those claims.

2. The problem in plain language

Orbital objects move quickly, and their future positions are estimated from observations and mathematical models. Two objects may pass near one another. Analysts need to estimate when and where closest approach occurs, and understand uncertainty in that estimate. A single nominal value may hide uncertainty in the input state, measurement, model or assumptions.

Problem example 1 — One number can hide uncertainty

Hypothetical example: a display says closest approach is 1.2 km. Without the epoch, coordinate frame, uncertainty, units and method, a viewer cannot tell how much to trust that value. OFDE should expose those details and show a justified range or distribution only if the code computes one.

Problem example 2 — Assumptions can change the result

Hypothetical example: two runs use slightly different initial state estimates or uncertainty assumptions. The closest-approach time or plausible separation range changes. OFDE should show the controlled input change and compare outputs side by side, rather than presenting a dramatic but unexplained risk badge.

Problem example 3 — An unexplained score is not decision support

Hypothetical example: an interface says “high risk” but does not reveal the formula, threshold, data or limitations. A judge cannot verify it. OFDE should define any project metric, show its inputs and units, and mark it exploratory if it has not been scientifically validated.

These are teaching examples, not real events, measured orbital data or outputs from a validated operational system.

3. Proposed solution: how OFDE addresses the examples

A. Traceable inputs

• Show source label (public or synthetic), object identifiers, epoch/time, units, coordinate frame where known, required fields and validation status.

• Reject missing or incompatible values with helpful errors; never silently invent inputs.

B. Documented calculations

• Explain the actual propagation and closest-approach method present in the code. If the feature is only illustrative, label it that way.

• Keep units, time systems, epochs and coordinate frames explicit. Do not mix incompatible data without conversion.

C. Controlled scenario comparison

• Run a baseline and alternative, changing one declared assumption at a time.

• Show comparable outputs, units, assumptions and a plain-language explanation of what changed.

D. Honest uncertainty

• Only display probability, confidence interval, uncertainty envelope or Monte Carlo distribution when the code and assumptions justify that specific quantity.

• A qualitative selector or drawn envelope must be labelled conceptual, not a statistically computed result.

E. Explainable outputs

• Define Threat Transfer, Future Exposure Index (FEI) and Decision Sensitivity as project-defined measures unless evidence establishes otherwise.

• For every metric, document formula, dimensions/units, normalization, thresholds, assumptions and limitations. Remove unsupported scores.

Example end-to-end workflow

Input: a labelled synthetic orbital-state sample with explicit epoch and units. **Baseline:** run the verified method. **Alternative:** change one stated assumption. **Compare:** show supported outputs side by side. **Explain:** identify the changed assumption and limitation. Never call a hypothetical result a real collision probability.

4. How it differs from existing approaches

Established conjunction-assessment and space-surveillance workflows already use orbital data, propagation, closest-approach analysis, uncertainty and expert review. Do not claim existing systems only consider one future or lack uncertainty analysis. The defensible distinction is the specific, transparent scenario-comparison experience actually implemented in this prototype.

Aspect	General existing workflows	OFDE intended contribution
Purpose	Estimate conjunction geometry and support expert assessment	Make scenario differences and sensitivity easier to inspect
Outputs	Depend on system; may include TCA, separation and uncertainty	Side-by-side scenario results with explicit assumptions
Explainability	Depends on tools and workflow	Show metric definitions, input provenance and why outputs changed
Validation	Operational systems vary; intended use may be validated	Research prototype; not operationally validated

Suggested pitch: “We are building an explainable research prototype that compares clearly defined orbital scenarios and exposes how assumptions affect displayed outcomes. We do not claim to replace operational conjunction assessment.”

5. Recommended software architecture

Logical layers

• **Data layer:** approved public data or offline synthetic fixtures; provenance, epoch, units, coordinate-frame metadata.

• **Validation layer:** schema checks, missing-field handling, supported formats and clear error messages.

• **Calculation layer:** tested functions for only the calculations actually supported by the code.

• **Scenario layer:** baseline and alternatives, with a record of the exact assumption changed.

• **Uncertainty/metrics layer:** only verified uncertainty methods and documented formulas for project-defined metrics.

• **Presentation layer:** input review, results, comparisons, plots, warnings, assumptions and report view.

• **Test layer:** unit tests, validation tests, integration tests and a repeatable offline demo.

Suggested screen flow

• 1. Show a clear “research prototype / not live operational data” banner.

• 2. Choose an included sample or load a supported file.

• 3. Review provenance, epoch, units, frame and input quality.

• 4. Run the documented calculation and show method/version.

• 5. Compare baseline and alternative scenarios.

• 6. Inspect metric formulas and uncertainty assumptions.

• 7. Produce a summary of inputs, method, outputs, warnings and limitations.

6. Step-by-step development plan

Step 0 — Audit first

Inspect index.html, style.css, script.js and calculation modules. List working, mocked, incomplete and tested features. Preserve working code.

Step 1 — Freeze scope

Choose one complete workflow. Avoid adding a database, authentication, live API or complex 3D unless required.

Step 2 — Define data schema

Agree on required fields, units, epochs, coordinate frame, missing-value policy and provenance. Include labelled synthetic fixtures.

Step 3 — Validate inputs

Reject missing, malformed or incompatible data instead of guessing.

Step 4 — Verify mathematics

Isolate propagation/TCA logic. Add known-answer or independently checked tests with documented tolerances.

Step 5 — Add scenario comparison

Compare baseline and controlled alternative; expose exactly what changed.

Step 6 — Handle uncertainty honestly

Inspect existing implementation. If Monte Carlo exists, test seed, sample count and convergence. Otherwise do not claim Monte Carlo.

Step 7 — Define metrics

Document formula, units, normalization, thresholds and limitations for Threat Transfer, FEI and Decision Sensitivity.

Step 8 — Connect UI to real outputs

Every numeric display must trace to input and computation; illustrative values must be labelled.

Step 9 — Test end-to-end

Run setup, type checks, unit/integration tests and the exact demo path. Record failures honestly.

Step 10 — Add fallback

Keep a known-good Git commit and local demo fixture so the project works offline.

Step 11 — Package

Include README, setup steps, data notes, assumptions, limitations, test report and demo script.

7. Round 2 when detailed rules are incomplete

The shared event screenshot states: top 50 Round 1 teams; 12-hour offline final; 24 October 2026; MITS Campus, Gwalior; evaluation on functionality, technical implementation, innovation, feasibility and demonstration. The Round 1 problem statement carries forward. Further instructions and deadlines will be sent to shortlisted teams. The screenshot does not state scoring weights, pitch duration, exact deliverables or permitted-tool rules.

Questions to ask organizers

• What must be submitted: source code, repository, PPT, report, video, deployed application or all of these?

• What are the scoring weights and presentation, demo and Q&A; time limits?

• Are AI coding assistants, internet access, external APIs and synthetic/offline datasets permitted?

• Will Wi-Fi, power, projector, display cables or hardware be provided?

• What are arrival/check-in time, submission deadline, fee deadline and venue instructions?

• Are there special aerospace-data, citation, safety or originality requirements?

Until clarified, build an offline-capable software MVP and avoid speculative features. Map each official requirement to a code item and test when the rules arrive.

8. Proposed 12-hour schedule

Time	Work	Exit condition
00:00–00:45	Read official instructions and freeze scope	Deliverables listed
00:45–01:30	Run baseline and save checkpoint	Known-good version starts
01:30–03:00	Verify core calculations	Core path and tests pass
03:00–05:30	Build highest-value missing feature	Feature works on documented inputs
05:30–06:00	Integrate and checkpoint	One shared version runs
06:00–08:00	Finish scenario comparison/explanation	Complete end-to-end workflow
08:00–09:15	Edge cases and regression tests	Critical cases pass
09:15–10:15	Polish demo/report	Outputs readable and reproducible
10:15–11:15	Rehearse and fix blockers	Demo succeeds twice
11:15–12:00	Final verification/submission	Required artifacts submitted

This is our internal plan, not the official event timetable. Follow organizer instructions if they differ.

9. Team responsibilities

• **Anuj Singh — lead/integration:** scope, architecture, merge decisions, end-to-end workflow, main pitch and integration.

• **Sarthak Gavnekar — orbital calculation validation:** inspect formulas, units, epochs, coordinate frames, TCA method and tests.

• **Nitin Chherke — uncertainty/scenarios:** verify implemented uncertainty method, scenario comparison, metric formulas and limitations.

• **Chirag Lalwani — UI/demo/QA:** user flow, controls, input validation, regression checks, demo fixtures and rehearsal.

This is a proposed split; adjust it to match each member's strengths. Everyone should understand the overall architecture.

10. Suggested 3–5 minute demo

• 0:00–0:30 — Explain the problem: a nominal prediction is difficult to interpret without assumptions and uncertainty.

• 0:30–1:00 — Identify the sample and state whether it is public or synthetic; show epoch and units.

• 1:00–2:00 — Run the supported analysis and explain it in plain language.

• 2:00–3:00 — Compare baseline and controlled alternative; state what changed.

• 3:00–4:00 — Show only the uncertainty/sensitivity that is implemented and tested.

• 4:00–5:00 — Explain the contribution, limitations and next steps.

Adjust to the official time limit. Keep the demo focused on one reliable workflow, not a tour of every screen.

11. Acceptance checklist

- A fresh checkout starts using documented commands.
- A known sample produces reproducible output.
- Every number has an input source, method, unit and interpretation.
- Invalid inputs fail visibly rather than generating plausible-looking outputs.
- Scenario comparisons show exactly which assumption changed.
- Random sampling, if present, is reproducible when required and has documented limitations.
- No unsupported live-data, probability, accuracy or operational claims appear.
- The main demo works offline using included fixtures.
- The team can explain calculations, tests, assumptions and limitations.
- All official submission requirements are met.

12. Risk and contingency plan

- **No internet/API:** use local fixtures, label them and rehearse offline.
- **Core calculation failure:** return to the last known-good commit and demonstrate only verified functionality.
- **Scope creep:** prioritize the must-have workflow and stop new features before final testing.
- **Integration conflict:** assign module owners, agree interfaces and commit small changes.
- **Rules unclear:** ask organizers rather than guessing pitch or tool rules.
- **Unsupported scientific claim:** remove it or label it conceptual until there is evidence.
- **Device/display issue:** keep a repository copy, chargers, local demo files, PDF deck and screenshots.

13. Master prompt for Claude

Copy this prompt into Claude and attach the existing source files or repository. Ask Claude to inspect the current implementation before modifying it.

You are a senior aerospace software engineer, scientific-computing reviewer, and full-stack developer helping our student team prepare for the Nakshatra Final Hackathon on 24 October 2026.

PROJECT
Name: Orbital Future Decision Engine (OFDE).
Team: Fantastic 4 — Anuj Singh, Sarthak Gavnekar, Nitin Chherke, Chirag Lalwani.
Problem statement: preserve the exact orbital debris/orbital traffic problem statement selected in Round 1. Do not replace it with a different problem.
Current state: we have a working prototype and website. Inspect the supplied files to determine which features are real, mocked, incomplete, and tested.

MISSION
Improve the EXISTING software into a reliable, explainable, offline-capable research prototype that compares clearly defined orbital scenarios and shows how assumptions affect outputs. Prioritize a complete end-to-end workflow, scientific integrity, reproducibility, tests and a judge-ready demo over visual complexity.

NON-NEGOTIABLE SCIENTIFIC INTEGRITY
1. Never invent live telemetry, orbital positions, collision probabilities, scores, experimental results, accuracy claims, citations or datasets.
2. Do not claim propagation, Monte Carlo, covariance propagation, conjunction probability or maneuver optimization exists unless you locate the implementation, verify the logic and add tests.
3. Label synthetic fixtures and hypothetical scenarios in the UI, code, reports and documentation.
4. Treat Threat Transfer, Future Exposure Index (FEI) and Decision Sensitivity as project-defined measures unless supplied evidence proves otherwise. Document formulas, units/dimensions, normalization, thresholds, assumptions and limitations. If unsupported, flag/remove the displayed score rather than fabricate it.
5. Do not present this prototype as operational collision-avoidance software or a replacement for established aerospace systems and expert review.
6. Never silently fill missing orbital inputs with invented values. Validate inputs and explain errors.
7. Preserve the exact Round 1 problem statement and core idea.

PHASE 1 — AUDIT
- List files, stack, commands, architecture, existing features and tests.
- Trace each visible metric to its real calculation and inputs.
- Identify mocked controls, hardcoded outputs, broken links, invalid assumptions, unit mismatches, missing validation and unsupported claims.
- Preserve working code; do not replace the project with a generic template.
- Report blockers, high-value improvements and optional improvements, then proceed with safe changes if you can edit the repository.

PHASE 2 — MVP WORKFLOW
A. Select an included sample or load a supported input file.
B. Show provenance (public vs synthetic), epoch/time, units, coordinate frame where known, required fields and validation.
C. Run only calculations genuinely supported by the code and documented method.
D. Compare a baseline with at least one controlled alternative scenario.
E. Explain what changed, which assumption changed and how to interpret the output.
F. Show uncertainty only if the method justifies it; otherwise label the visual as conceptual.
G. Provide a summary containing inputs, assumptions, method/version, outputs, warnings and limitations.
H. Work offline with included demo fixtures.

PHASE 3 — ENGINEERING
- Follow the current stack unless a change is justified.
- Separate UI, validation, domain calculations, scenario orchestration, metric definitions and report generation.
- Use explicit types/interfaces and units.
- Make calculations deterministic where feasible. If randomized sampling exists, support a fixed seed for reproducible tests.
- Add tests for input validation, math functions, scenario comparisons, failure cases and the end-to-end happy path.
- Prefer known-answer or independently checked tests; document tolerances and assumptions.
- Label chart axes, units and demo status.
- Make each control functional or remove/disable it honestly.
- Avoid unnecessary dependencies, paid services, API keys, databases, authentication or cloud deployment unless requirements demand them.
- Make small, reviewable changes with checkpoints.

PHASE 4 — SCIENTIFIC REVIEW
Inspect whether the project truly implements orbital state parsing and epoch handling, frame/unit conversions, propagation, time of closest approach, relative separation, uncertainty/covariance treatment, Monte Carlo (if claimed), and the three project-defined metrics.
For each verified calculation, document equations/algorithm, inputs, output units, assumptions, test cases and limitations. If a method cannot be verified, mark it unverified. Do not invent a collision-probability method.

PHASE 5 — HACKATHON RELIABILITY
- Add clear setup/start/test instructions.
- Include offline demo fixtures with provenance labels.
- Provide a single demo path that can be rehearsed twice.
- Record commands actually run and real test results; do not claim unrun tests passed.
- State environment limitations if dependencies or commands cannot be run.
- Keep the app usable when network resources fail.
- Ensure responsive layout, accessibility and readable projector output.

PHASE 6 — DELIVERABLES
Create/update:
1. Source files, preserving filenames where practical.
2. README with setup, run, test, demo and troubleshooting.

3. ARCHITECTURE.md describing modules and data flow.
4. SCIENTIFIC_METHOD.md documenting verified methods, formulas, units, assumptions, sources and limitations.
5. TEST_REPORT.md listing commands actually run, results, failures and untested areas.
6. DEMO_SCRIPT.md for a 3–5 minute demo, adaptable to official time limits.
7. Prioritized remaining work and risks.
8. A concise summary of significant code changes.

WORKING RULES
- Inspect first; do not assume.
- Do not silently remove useful functionality or change the Round 1 problem statement.
- Do not use “rest unchanged” or omit necessary code.
- Never claim a build, test, API, calculation or feature succeeded unless it actually did.
- If you cannot run commands or install dependencies, state exactly what was not verified and give commands for our local Windows environment.
- Ask only questions that block progress; otherwise make conservative, documented choices.
- Prioritize correctness and a reliable demo over extra features.

Start by inspecting the supplied repository and reporting the actual project state. Then implement the highest-priority safe improvements.

14. Final recommendation

Audit and validate the existing prototype before adding features. Build one reproducible workflow and make its outputs explainable. When the official organizer instructions arrive, map every requirement to a task and test. The best demo is the one the team can explain and verify—not the one with the most impressive unsupported claims.

Prepared for Fantastic 4. Event facts above are based on the screenshot shared by the team; technical details are planning guidance and must be checked against the actual code and official rules.